LINUX · NVIDIA · RUST

llmpager

Running Mixture-of-Experts models bigger than VRAM

A 30B-parameter MoE, 18.5 GB of weights, serving two models at once —
on a 16 GB consumer GPU, with experts paged from NVMe.

Glenn West · August 2026

Agenda

Overview

The problem, the idea, the hardware

Source

Architecture, formats, pager, kernels

Results

Every number measured, kept or rejected

Lessons

What the data taught us

Everything in this deck is reproducible: numbers come from `docs/BENCHMARKS.md` and `docs/PERFORMANCE.md` in the repo, measured on one machine in a single ~5-hour session.

OVERVIEW

The problem: MoE weights don't fit

- Mixture-of-Experts models activate a **tiny fraction** of their weights per token — Qwen3-30B-A3B touches ~3B of 30B parameters.
- Conventional engines still require **all** expert weights resident in GPU or CPU memory.
- Our card: RTX 5060 Ti, **16 GB VRAM**. The model: **18.5 GB** at 4-bit — before KV cache and activations.

> *Expert routing is heavily skewed in practice.*
> *Skew is what caches are for.*

> **In plain terms:** the model is 18.5 GB of numbers; the graphics card holds 16 GB. Normally that's game over — you buy a bigger GPU. But per word generated, the model only *uses* about a tenth of itself.

Qwen3-30B-A3B	
Layers	48
Experts per layer	128
Active per token	top-8
Expert weights (q4)	15.4 GB
Shared core (bf16)	3.1 GB
Total vs 16 GB VRAM	**18.5 GB**

OVERVIEW

The vocabulary, in plain terms

Term	What it means here
Token	A word-piece. Models read and write ~1 token at a time; "tok/s" is roughly writing speed. 40 tok/s is faster than most people read.
Weights	The billions of learned numbers that *are* the model. They must be somewhere the processor can reach to generate anything.
VRAM	The GPU's own fast memory — small (16 GB here) but ~20× faster than main RAM for this work.
Mixture of Experts	Instead of one giant network, 128 small specialist networks ("experts") per layer. A "router" picks the best 8 for each token — so ~90% of the model sits idle at any moment.
Quantization (q4)	Storing weights with 4 bits instead of 16 — like compressing a photo. 4× smaller, tiny loss in fidelity (we measured it: slide on perplexity).
Paging / cache	The desk-and-filing-cabinet trick every OS uses: keep what you're using on the desk (VRAM), fetch the rest from the cabinet (disk) when asked, keep recently used items handy.
Perplexity	The standard quality score for language models: how "surprised" the model is by real text. Lower is better — and any corruption of the weights shows up in it immediately, which makes it our correctness detector.

OVERVIEW

The idea: treat experts like pages

- Keep the **shared core** resident in VRAM: embeddings, attention, router, norms, and the KV cache (the model's running notes on the conversation so far).
- **Stream routed experts from NVMe on demand**; cache the hot ones in VRAM with an aged-LFU, per layer.
- Inspired by turbo-fieldfare (Apple Silicon / Metal); rebuilt for Linux + CUDA, then taken further: multi-model serving with a VRAM budgeter.

Hits cost nothing. Misses overlap with compute. The disk only ever sees what the cache can't hold.

> **In plain terms:** a small desk and a big filing cabinet. The always-needed pages stay on the desk; specialists are fetched from the cabinet only when called, and the ones called often just stay out. The trick is fetching the next folder *while* you're still reading the current one.

The lab: one VM, measured limits

ai.g8.lo (VM 600 on Proxmox)	
GPU	RTX 5060 Ti 16 GB (Blackwell), vfio passthrough
CPU / RAM	12 vCPU · 64 GB
Storage	200 GB root + 800 GB /data (NVMe-backed)
Stack	Debian 13 · driver 610.43 · CUDA 13.3 · Rust 1.96 (deb/rpm packages for Debian & Fedora)

Day-zero war story: the new motherboard's ACPI D3cold support wedged the GPU under vfio — `vfio_pci.disable_idle_d3=1` (Proxmox bug #7374) fixed passthrough before line one of code.

Measured ceiling	Value
VRAM bandwidth (spec)	~448 GB/s
Pinned H2D over PCIe	25.3 GB/s
NVMe O_DIRECT random 3 MB reads	4.3 GB/s
Expert fetch latency (99%)	< 2 ms

Every design decision traces back to these four numbers.

> **In plain terms:** these are the speed limits of each road — GPU memory is a motorway (448), the GPU-to-computer link a main road (25), the disk a side street (4.3). The whole design is about keeping traffic off the side street.

SOURCE

Five crates, ~5k lines of Rust + CUDA C

Crate	What it owns	Depends on GPU?
`llmpager-core`	.llmpk pack format · aged-LFU cache · q4g64 quantization · safetensors reader	No — tests run anywhere
`llmpager-cuda`	Driver wrapper (runtime-loaded libcuda) · async pager · 12 CUDA kernels	Runtime only, no toolkit
`llmpager-convert`	HF checkpoint → expert pack + pageable core (21 s for 30B)	No
`llmpager-run`	Decode loop · KV cache · router · perplexity mode · CLI	Yes
`llmpager-serve`	OpenAI-compatible HTTP · model registry · VRAM budgeter · systemd	Yes

- **No CUDA toolkit at runtime** — the driver API is loaded from `libcuda.so.1`; kernels compile to PTX at build time and the driver JITs them onto Blackwell.
- Apache-2.0 · **github.com/glennswest/llmpager** · tagged releases with **.deb / .rpm packages** — `apt install` on Debian, `dnf install` on Fedora, then edit one config file and `systemctl start llmpager`.

> **In plain terms:** five building blocks — a file format, a librarian that moves weights, a converter that repackages downloaded models, a runner that generates text, and a web server that answers requests like any AI API.

SOURCE

The expert pack: .llmpk

```
[0..4096)   header: magic, JSON meta
            (model config embedded)
[4096..)    index: offset+len per
            (layer, expert) — 16 B each
[...]       blobs: gate/up/down weights,
            q4g64, each 4096-aligned
```

q4g64: symmetric 4-bit, groups of 64, f16 scale per group — scales-then-nibbles layout so the GEMV kernel streams both linearly.

- **Everything is 4096-byte aligned** so every read can use O_DIRECT — reads that bypass the OS's file cache — with aligned offset, buffer, and length as the disk requires.
- One blob per (layer, expert): the unit of paging, ~2.5 MB for this model.
- The resident core ships separately as ordinary safetensors — **pageable as one artifact**, which is what makes 0.8 s model switching possible later.
- Converter reads HF shards directly with `pread` — no 4 GB shard ever loaded whole; 48×128 experts quantized in parallel: **21 seconds**.

> **In plain terms:** a carefully organized archive where every expert sits at a known position, sized in disk-friendly chunks — so any expert can be grabbed with one clean read, no unpacking.

SOURCE

The async pager

- **I/O worker pool**: each worker owns an O_DIRECT fd, a pinned staging buffer, and a CUDA stream.
- Miss path: `pread → pinned → cuMemcpyHtoDAsync → cuEventRecord`. Compute streams wait on the event — **no host blocking on hits**.
- Cache slots are **pinned by refcount** while kernels read them; releases ride a CUDA-event ring, not stream syncs.
- Slot memory is **per-layer arenas** — thousands of individual 2.5 MB allocations wasted ~60% of VRAM to allocation granularity (found as an OOM).

Pager property	Measured
Paging ceiling, 48 slots + prefetch	113 tok/s
Prefetch one layer ahead	+42%
Fetch latency, 99th percentile	< 2 ms
Stalls observed, all runs	0

The ceiling means: the pager can feed experts 3× faster than the compute currently consumes them.

> **In plain terms:** a team of librarians. Ask for 8 folders; whatever's on the desk is handed over instantly, the rest arrive from the cabinet in under 2 ms — while the GPU keeps working on something else. Nobody ever stands around waiting.

SOURCE

Twelve kernels, all verified against CPU references

- q4g64 GEMV — matrix×vector multiply, *the* operation of generating a token (single + **batched across top-k experts**)
- bf16 GEMV (vectorized `uint4` loads)
- RMSNorm (row-batched for per-head q/k norm)
- NeoX RoPE · GQA decode attention · KV append
- SiLU-mul · residual add · weighted MoE reduce · embed gather

> *Worst relative error across the whole set: **1.9e-6**.*
> *Correctness first made every optimization a pure perf diff.*

> **In plain terms:** "kernels" are the small programs that run on the GPU and do the actual arithmetic. Each one was checked against a slow-but-trusted CPU version to parts-per-million agreement before we made anything fast.

```
// build.rs: nvcc → PTX (compute_80),
// driver JITs to sm_120 at load
q4g64_gemv<warp per row>
  scales: f16, streamed linearly
  data:   8×u32 nibble words / group
  x:      float4 pairs
// 35 → 122 GB/s from vector loads,
// → 138 GB/s from fp16 magic unpack
```

RESULTS · HOUR THREE, 19:00

"The capital of France is

Paris. The capital of the United Kingdom is London..."

First real tokens from an 18.5 GB model on a 16 GB card —
converter → pack → kernels → router → pager, all agreeing at once.

21s
to convert the 30B checkpoint

83%
expert-cache hit rate, first run

~3h
from wedged GPU + empty repo to this

RESULTS

Decode speed: 19.9 → 41.0 tok/s (+106%)

- Vectorized loads: q4 GEMV 35 → 122 GB/s; bf16 GEMV to ~VRAM bandwidth. **+56% alone.**
- RAM tier: pack (15.4 GB) fits in RAM (64 GB) — misses become memory copies. Cold prompts: **20.4 → 32.2 tok/s.**

> **In plain terms:** 41 tokens/second is ~30 words a second — comfortably faster than you read. Each bar is one specific engineering change; nothing here is a bigger GPU or a smaller model.

RESULTS

Cache size is a curve, not a cliff

Slots/layer	VRAM	Warm prompt	Cold prompt
24	2.9 GB	14.8 tok/s	11.2
32	3.9 GB	21.0	13.2
48	5.8 GB	33.3	20.4
64	7.7 GB	34.6	25.5

O_DIRECT, Qwen3-30B-A3B, 64 tokens, hit rates 58–90%.

- Real routing is **flatter than synthetic benchmarks** — the 80/20 assumption gave 93% hits; reality gave 84–89%.
- Warm workloads plateau at 48 slots; cold workloads keep paying for misses.
- Conclusion that shaped the roadmap: **make misses cheaper, not just rarer** — which is exactly what the RAM tier did.

> **In plain terms:** a bigger desk means fewer trips to the filing cabinet — but past a point, extra desk space barely helps. The better move was making each trip 6× faster.

RESULTS

Correctness, proven: paging is lossless

Configuration	Perplexity
48 slots, bf16 core	**8.6199**
24 slots, bf16 core	**8.6199**
48 slots, q4 core	9.0038 (+4.5%)

Teacher-forced NLL, 137-token English text, `--ppl` mode.

- Identical to four decimals across cache sizes: **the cache changes when weights arrive, never what they are.**
- This invariance is what makes the VRAM budgeter safe — resizing a live model's cache carries zero quality risk.
- The q4-core row prices an earlier rejected experiment: +4.5% PPL was the quality tax, on top of being slower in decode.

> **In plain terms:** perplexity measures how "surprised" the model is by real text — lower is better, and it's exquisitely sensitive to any corruption. Identical scores at different cache sizes means paging changes *when* weights arrive, never the answers.

RESULTS

Two models, one 16 GB card, live service

- OpenAI-compatible endpoint (systemd, port 8090); requests routed by the standard `model:` field.
- Qwen3-30B-A3B + Qwen3-Coder-30B-A3B — **36.9 GB of weights, both warm at once**.
- VRAM budgeter: solo model gets 48 slots (~33 tok/s); when a second warms, residents shrink to 24/24 (~14 tok/s each).
- Unload actually frees VRAM (pager arenas, core, KV, events) — eviction is honest.

```
# journalctl -u llmpager
loaded qwen3-30b-a3b (48 slots) in 2.4s
budgeter: qwen3-30b-a3b 48 → 24 slots
loaded qwen3-coder (24 slots) in 0.8s
```

> ***0.8 seconds*** *to bring an 18.5 GB model into rotation — only the 3.1 GB core moves; experts page themselves in.*

> **In plain terms:** like two apps sharing your laptop's memory — the one you're using gets more, the idle one shrinks, and switching is instant because most of each app was never "loaded" in the old sense to begin with.

LESSONS LEARNED

1 · Measure, don't guess — three confident ideas, three rejections

Plausible idea	What the data said	Why
Quantize the core to q4 (4× less bandwidth!)	**25.9** vs 33.1 tok/s + quality drift	The q4 kernel is unpack-ALU-bound at 122 GB/s; bf16 runs at ~400. Bandwidth ratios only convert to speed if kernels are equally efficient.
Prefetch layer L+1 with layer L's experts	**10.8** vs 18.5 tok/s, 3× disk traffic	Qwen3 routing is uncorrelated across layers. Wrong prefetch is worse than none: it evicts hot entries *and* steals bandwidth.
More I/O threads = more disk throughput	8 threads: 3.6 GB/s · 1 thread: **4.3**	One virtio-scsi queue. Few deep readers beat many shallow ones.

> *Same weights, different workload, opposite conclusion: q4-core loses in decode but wins in teacher-forced PPL mode (lm_head-dominated).* ***Measure the workload you ship.***

2 · The memory hierarchy is the product

- **O_DIRECT is a policy, not a principle.** Bypassing the page cache is right when the pack exceeds RAM — and exactly wrong when it fits. Flipping one flag (`--direct=0`) was the single biggest decode win (+58% on cold prompts).
- **Allocation granularity is real money.** 3,072 individual 2.5 MB device allocations silently became ~4 MB each — 60% of the cache budget gone. Arena allocation fixed an "impossible" OOM.
- **Overlap beats bandwidth.** One layer of prefetch bought +42% with zero extra bytes moved. The event-ring release bought +6% by never draining the pipeline.
- **Caches must age.** Pure LFU pins yesterday's hot experts forever; halving counters periodically lets the cache track the workload's drift.
- **The bottleneck migrates.** Disk → kernels → launch overhead → misses → host round-trips. Every fix moved it; only measurement found where it went.

3 · Process: correctness first, ship the ladder

✔ **Correctness before speed.** Every kernel verified against a CPU reference (worst 1.9e-6) before any optimization — so every perf change was a pure diff against known-good output.

✔ **Prove the ceiling first.** M0/M1 measured a 113 tok/s paging ceiling before any model code existed — we always knew paging wasn't the wall.

✔ **Negative results are deliverables.** Three rejections live in PERFORMANCE.md with numbers; they're worth as much as the wins.

Milestone ladder	Releases
M0 paging proof → M1 async pager	hour 1
M2 real tokens ("Paris.")	hour 2–3
M3 perf: +106%	hour 3–4
M4 serving · M5 multi-model	hour 4–5
Total	**10 releases, one evening**

WHAT'S NEXT

Headroom and horizons

Near term

- Chase the ceiling: 41.0 measured vs 113 tok/s paged — GPU router top-k, warm-start prefill, fp16 KV.
- Budgeter v2: request-rate-weighted cache sizing.
- 80B-A3B coder: same 3B active → same speed class, ~40 GB pack — a "needs 45 GB VRAM" model on 16 GB.

Kimi-class (1T+), sized honestly

- ~600 GB q4 pack fits the 800 GB /data volume.
- 16 GB VRAM caches only ~1.3% of 23k experts → the **pinned partial RAM tier** becomes load-bearing.
- Expected: 0.5–1.5 tok/s — batch territory, same class as published demos. Interactive stays the 3B-active sweet spot.

> *The machinery is model-agnostic: pack + core + config. Anything MoE with skewed routing pages.*

TRY IT

One curl away

```
curl http://ai.g8.lo:8090/v1/chat/completions \
  -d '{"model": "qwen3-coder-30b-a3b",
       "messages": [{"role":"user","content":"..."}]}'
```

github.com/glennswest/llmpager **docs/PERFORMANCE.md — every number** **docs/DESIGN.md — the architecture**

An 18.5 GB model answering questions about memory paging,
while its own experts are being paged. Thank you.